PARAKH — Stage 3 PRD

Semantic Layer & Peer Cell Formation

1. Objective

Build a system that transforms raw records into **meaningful comparison groups (peer cells)**.

Each record must be assigned to a **peer cell**:

$$
(k, s)
$$

Where:

- (k): semantic cluster (type of work)
- (s): cost stratum (scale of work)

These peer cells are the **foundation for all downstream anomaly detection**.

---

2. Scope

Included

- Text embedding of `work_name`
- Semantic clustering
- Cost stratification
- Peer cell assignment
- Duplicate candidate detection (pairwise scoring)

Excluded

- Risk scoring
- Confidence scoring (already done)
- Entity graph construction

---

3. Inputs

From Stage 1:

```
Corpus.records
```

From Stage 2:

```
ConfidenceResult (optional for filtering low-C records)
```

---

4. Outputs

Each record must include:

```
{
  "cluster_id": int,     # k
  "cost_stratum": int,   # s
  "peer_cell": (k, s),
  "embedding": vector,
  "duplicate_score": float
}
```

---

5. Semantic Embedding

5.1 Input Field

- `work_name` (primary)
- optionally concatenate:
    - `implementing_agency`
    - `district`

5.2 Embedding Model

Use:

- Sentence transformer (multilingual)

Examples:

- `all-MiniLM-L6-v2` (fast baseline)
- multilingual model preferred (code-mixed data)

5.3 Preprocessing

Before embedding:

- lowercase
- remove punctuation

- normalize whitespace
- remove boilerplate words:
    - "construction of"
    - "development of"

5.4 Output

```
embedding.shape = (d,)   # e.g., d = 384 or 768
```

---

6. Clustering (Semantic Groups)

6.1 Algorithm

Use:

```
HDBSCAN
```

Reason:

- No need to predefine number of clusters
- Handles noise
- Works with uneven cluster sizes

6.2 Parameters

- `min_cluster_size = 20`
- `metric = cosine`

6.3 Output

Each record gets:

```
cluster_id = k
```

Noise points:

- Assign `cluster_id = -1`
- Handle separately

6.4 Post-processing

- Remove clusters with size < threshold
- Merge very small clusters into nearest large cluster

---

7. Cost Stratification

7.1 Input

- `sanction_amount`

7.2 Transformation

$$
x = \log(a + 1)
$$

7.3 Stratification

Option A (recommended):

- Quantile bins (e.g., 5 bins)

Option B:

- Fixed log-scale buckets

7.4 Output

```
cost_stratum = s
```

---

8. Peer Cell Formation

Each record:

```
peer_cell = (cluster_id, cost_stratum)
```

8.1 Minimum Cell Size

If:

```
len(cell) < n_min
```

Then:

- mark as **unstable**
- exclude from downstream scoring

Default:

```
n_min = 15
```

---

9. Duplicate Detection (Candidate Scoring)

9.1 Pairwise Score

$$
D(i,j) =
\cos(e_i, e_j)
\cdot \mathbb{1}[d_i = d_j]
\cdot \exp\left(-\frac{|t_i - t_j|}{\tau_d}\right)
$$

9.2 Parameters

- (\tau_d = 180) days

9.3 Efficient Computation

Avoid (O(N^2)):

- Only compare within:
    - same district
    - same cluster

9.4 Per-record Score

$$
D_{\max}(i) = \max_{j \neq i} D(i,j)
$$

9.5 Output

```
duplicate_score = D_max
```

10. Implementation Architecture

10.1 Modules

- `embedding.py`
- `clustering.py`
- `stratification.py`
- `peer_cells.py`
- `duplicate_detection.py`

10.2 Core Class

```
class SemanticLayer:
    def fit_transform(self, corpus):
        return SemanticResult
```

10.3 Output Object

```
class SemanticResult:
    cluster_ids: List[int]
    cost_strata: List[int]
    peer_cells: List[Tuple]
    embeddings: np.ndarray
    duplicate_scores: List[float]
```

11. Non-Functional Requirements

Performance

- 50k records processed in < 10 seconds

Memory

- Embeddings stored efficiently (float32)

Determinism

- Fix random seed for clustering

---

12. Validation & Testing

12.1 Cluster Quality

Check:

- similar work_names grouped together
- unrelated works separated

12.2 Cell Distribution

```
print(cell_sizes.describe())
```

Ensure:

- no extreme imbalance
- reasonable spread

12.3 Duplicate Sanity Check

- identical names → high score
- unrelated → low score

---

13. Edge Cases (MANDATORY)

Handle:

- Empty or short `work_name`
- All records same text
- Extremely skewed cost distribution
- Single cluster output
- All noise from HDBSCAN

---

14. Acceptance Criteria

Stage complete if:

- ☐ Every record has cluster_id
- ☐ Cost strata assigned

- ☐ Peer cells formed
- ☐ Duplicate scores computed
- ☐ Small cells handled correctly

---

15. Definition of Done

- Records grouped into meaningful peer cells
- Ready for Stage 4 (Risk Signals)

---

16. Key Design Principle

> **Compare like with like.**

Anomaly detection without proper peer grouping is mathematically invalid.

This stage ensures:

- roads compared to roads
- small works compared to small works

—not everything against everything.
